latent-play Architecture: Embeddings, Unsupervised LAM, JEPA + SIGReg World Model & Dual Action Loops

STAGE 1: What is Embedded & How the Latent Action Model (LAM) Discovers Discrete Actions (100% Label-Free Video)

Consecutive Frames
$x_t, x_{t+1} \in [0,1]^{3\times 64\times 64}$

Raw RGB video clip
(No action labels a_t)

→

PatchEmbed (8x8) + STBlocks
(Inverse Dynamics E_{IDM})

Conv2d(3->D, k=8, s=8) -> 64 patches
Temporal diff $\Delta h_t = h_{t+1} - h_t$
+ Spatial Motion Attention Pool

$u_t \in \mathbb{R}^{16}$ →

Cosine Vector Quantizer
Codebook $C \in \mathbb{R}^{8\times 16}$

$k_t = \arg\min_k \|\hat{u}_t - \hat{c}_k\|_2$
Outputs discrete code $k_t \in \{0..7\}$
+ Batch Entropy Reg (8/8 active)

$z_q(k_t) + x_t$ →

Forward Decoder (D_{FDM})
Residual Pixel Head

Takes current x_t + code $z_q(k_t)$
Predicts $\hat{x}_{t+1} = x_t + \Delta\hat{x}_t$

→

Stage 1 Loss
$\mathcal{L}_{\text{LAM}}$

Motion-Weighted MSE
$w = 1 + 5|x_{t+1} - x_t|$
$+\mathcal{L}_{\text{VQ}} + 0.1\mathcal{L}_{\text{ent}}$

Learnable in Stage 1: Inverse Encoder E_{IDM}, Codebook C ($K=8$), Forward Decoder D_{FDM} (1.68M params). After Stage 1, LAM is FROZEN to label all video transitions with $k_t \in \{0..7\}$.

STAGE 2: How the JEPA Predictor (g_ψ) is Trained & Exactly Where SIGReg + VICReg Act to Prevent Collapse

★ SIGReg + VICReg ACT HERE (Directly on Online Encoder Output z_t)

Pools $\bar{z}_t \in \mathbb{R}^{192}$ across the batch:
(1) SIGReg ($\lambda = 0.05$): $M = 64$ random 1D projections $u^{(m)}$ + Epps-Pulley test $\rightarrow \mathcal{N}(0, I_{192})$
(2) VICReg Hinge ($\lambda = 0.05$): Forces per-dimension Std($\bar{z}_t$) ≥ 1.0

$\bar{z}_t$ ↑

Action Embedding Table E_a [TRAINABLE 8×192]

Frozen Stage-1 LAM provides discrete code $k_t \in \{0..7\}$
Looks up vector $e(k_t) \in \mathbb{R}^{192}$ & adds to all 64 spatial tokens of z_t

$e(k_t)$ ↓

Current Frame x_t
PatchEmbed (8x8)

Splits 64×64 into 64
patches (8×8) + PosEmbed

→

Online ViT-S/8 (f_θ)
[TRAINABLE via Backprop]

4 Transformer Blocks
Outputs $z_t \in \mathbb{R}^{64\times 192}$

$z_t \in \mathbb{R}^{64\times 192}$ →

Causal ST-Predictor g_ψ [TRAINABLE World Model]

2 Spatio-Temporal Blocks (Spatial Attn + Causal Time Attn)
Predicts residual transition: $\hat{z}_{t+1} = z_t + g_\psi(z_{0..t}, e(k_{0..t}))$

$\hat{z}_{t+1}$ →

Stage 2 Loss
$\mathcal{L}_{\text{JEPA}}$

Latent L_1 Prediction:
$\|\hat{z}_{t+1} - \text{sg}[z^{\text{tgt}}_{t+1}]\|_1$

$+0.05\,\mathcal{L}_{\text{SIGReg}}(\bar{z}_t)$
$+0.05\,\mathcal{L}_{\text{var}}(\bar{z}_t)$

Backprop updates:
$f_\theta,\ E_a,\ g_\psi$

EMA $\tau = 0.996$ ↓

Next Frame x_{t+1}
PatchEmbed (8x8)

Future observation
at step $t+1$

→

Target ViT-S/8 ($f_{\bar{\theta}}$) — [STOP-GRAD] (No Backprop)

Outputs target representation $z^{\text{tgt}}_{t+1} = \text{sg}[f_{\bar{\theta}}(x_{t+1})] \in \mathbb{R}^{64\times 192}$
Weights updated purely via Exponential Moving Average: $\bar{\theta} \leftarrow 0.996\,\bar{\theta} + 0.004\,\theta$

$\text{sg}[z^{\text{tgt}}_{t+1}]$ →

STAGES 3 & 4: Dual Action Loops at Test Time — Inner Mental Loop (CEM in JEPA) vs. Outer Physical Loop (MPC in Game)

1. Outer Physical Loop
(Crafter / Grid Env)

Agent observes live frame x_t
and target visual goal x_{goal}

Encodes both via frozen f_θ:
$z_t = f_\theta(x_t),\ z_{\text{goal}} = f_\theta(x_{\text{goal}})$

z_t, z_{goal} →

2. INNER MENTAL LOOP: CEM in Latent Space (4 Iters x 128 Rollouts)

Sample & Rollout in g_ψ

Samples $M = 128$ action sequences
$k^{(m)} = (k_0, \ldots, k_5) \in \{0..7\}^6$
Unrolls 6 steps in latent space:
$\hat{z}^{(m)}_{h+1} = \hat{z}^{(m)}_h + g_\psi(\hat{z}^{(m)}_h, E_a(k^{(m)}_h))$

→

Score & Refine Top-16 Elites

Computes latent L_1 distance to goal:
$\text{Cost}^{(m)} = \|\hat{z}^{(m)}_6 - z_{\text{goal}}\|_1$
Updates $P^{(i+1)}$ from Top-16 elites
After 4 iters -> picks best $k^*_0 \in \{0..7\}$

$k^*_0 \in \{0..7\}$ →

3. Few-Shot Action Alignment (P_ϕ)
[Trained on 0.1% - 10% Labelled Pairs]

2-Layer MLP takes $(k^*_0, \bar{z}_t)$ and translates
latent code to real game control ($A = 17$)
with log-prior class-imbalance correction:
$\hat{a}_t = \arg\max_a [\log P_\phi(a|k^*_0, \bar{z}_t) - 0.5\log P(a)]$

4. Outer MPC Feedback Loop: Execute $\hat{a}_t$ in Crafter -> Environment steps to x_{t+1} -> Re-plan next step until $x_t = x_{\text{goal}}$